Python Activity-Based Learning Assignment

Ten selected case studies

Student Name: _____

Roll Number: _____

Branch / Section: _____

Course: Python Programming

Dr. RVR & JNR Institute of Technology

Case Study 1: ATM Transaction Sequence Simulation (#13)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Simulate deposits and withdrawals in sequence, maintain account balance, reject invalid transactions and report a transaction summary.

Objectives

Use variables, conditions, loops, a list and tuples to process and record transactions.

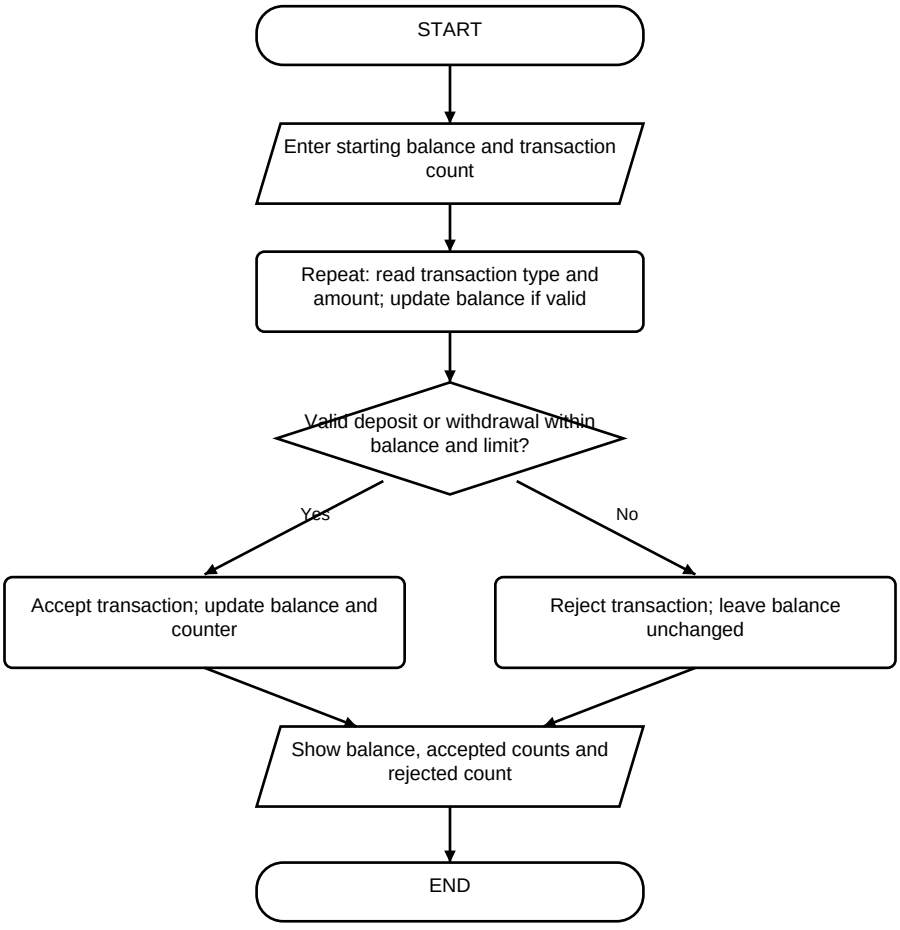
Assumptions

Starting balance is non-negative; transaction amounts are positive whole rupees; one withdrawal is limited to Rs. 10,000; withdrawals cannot exceed available balance.

Algorithm

Read and validate starting balance; read transaction count; for each transaction validate amount and type; update balance and counters only for accepted transactions; print summary and records.

Flowchart



Case Study 1: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
INPUT balance; repeat while balance < 0
INPUT transaction_count; set counters to zero
REPEAT transaction_count times: INPUT type, amount
IF amount <= 0: reject
ELSE IF type is deposit: add amount; count deposit
ELSE IF type is withdraw AND amount <= 10000 AND amount <= balance: subtract; count withdrawal
ELSE reject
DISPLAY balance and counters
```

Test Cases

1) Start 5000; deposit 2000; withdraw 1500 -> final 5500, 2 accepted.

2) Start 1000; withdraw 1500 -> rejected; balance 1000.

3) Start 20000; withdraw 12000 -> rejected by limit.

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

The simulation uses sequence processing and conditional validation to maintain an accurate account balance.

Case Study 1: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("ATM Transaction Sequence Simulation")
balance = int(input("Starting balance: "))
while balance < 0:
    print("Balance cannot be negative")
    balance = int(input("Starting balance: "))
count = int(input("Number of transactions: "))
deposits = 0
withdrawals = 0
rejected = 0
records = []
for i in range(count):
    kind = input("Type (deposit/withdraw): ").lower()
    amount = int(input("Amount: "))
    if amount <= 0:
        print("Rejected: amount must be positive")
        rejected += 1
        records.append((kind, amount, "Rejected"))
    elif kind == "deposit":
        balance += amount
        deposits += 1
        records.append((kind, amount, "Accepted"))
    elif kind == "withdraw" and amount <= 10000 and amount <= balance:
        balance -= amount
        withdrawals += 1
        records.append((kind, amount, "Accepted"))
    else:
        print("Rejected: invalid type, limit, or balance")
        rejected += 1
        records.append((kind, amount, "Rejected"))
print("Final balance:", balance)
print("Deposits:", deposits, "Withdrawals:", withdrawals, "Rejected:", rejected)
for record in records:
    print(record)
```

Case Study 2: Repeated Transaction Detection (#28)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Flag records with a previously used transaction ID or an amount above a supplied threshold.

Objectives

Store records as tuples; use nested loops and conditions to identify suspicious entries.

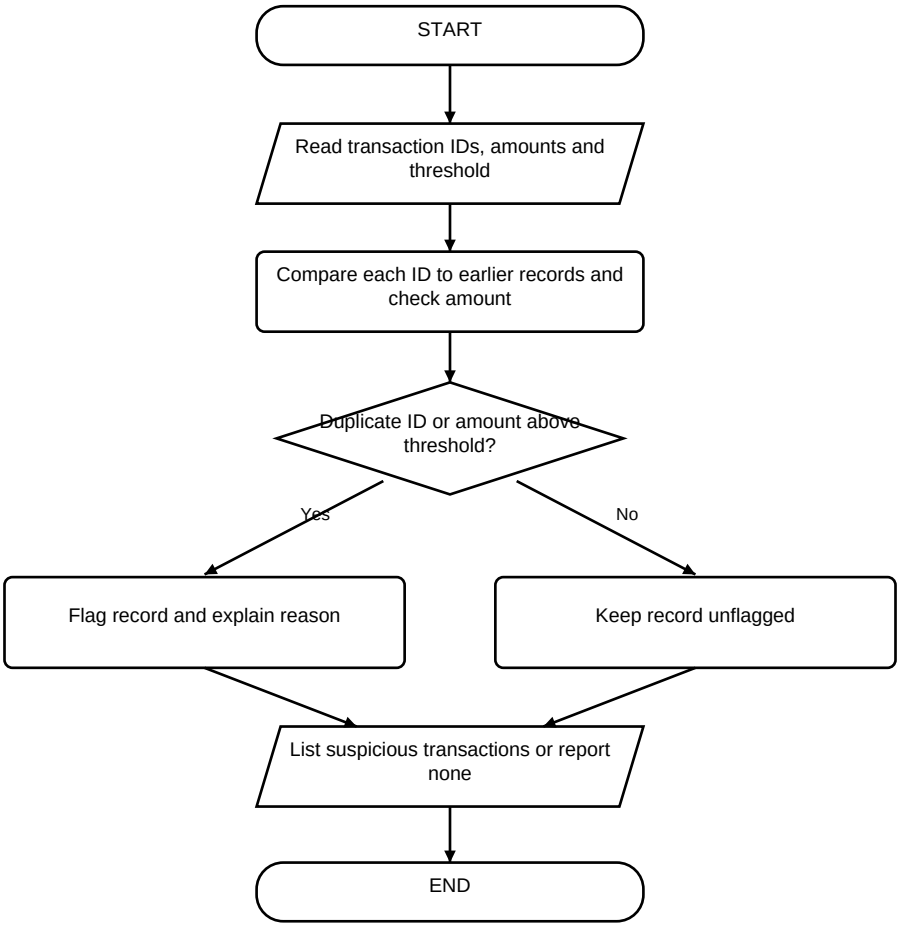
Assumptions

IDs are strings; amounts and threshold are non-negative; repeated occurrences after the first are duplicates.

Algorithm

Read records and threshold; compare each ID against earlier IDs; flag repeated IDs or high amounts; display results.

Flowchart



Case Study 2: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
INPUT n; repeat n times: INPUT ID and amount; append tuple
INPUT threshold
FOR each record i: duplicate = false
FOR each earlier record j: if IDs match, duplicate = true
IF duplicate OR amount > threshold: display record and reason
```

Test Cases

A1:100, B2:200, A1:50; threshold 150 -> B2 high, second A1 duplicate.

X1:40, X2:50; threshold 100 -> none.

Q7:300, Q7:400; threshold 250 -> both high; second also duplicate.

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

Nested loops detect duplicate identifiers while a threshold check finds unusually large amounts.

Case Study 2: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("Repeated Transaction Detection")
n = int(input("Number of transactions: "))
records = []
for i in range(n):
    tid = input("ID: ")
    amount = float(input("Amount: "))
    records.append((tid, amount))
limit = float(input("Suspicious amount threshold: "))
found = False
for i in range(len(records)):
    tid, amount = records[i]
    duplicate = False
    for j in range(i):
        if tid == records[j][0]:
            duplicate = True
    if duplicate or amount > limit:
        found = True
        if duplicate:
            reason = "duplicate ID"
        else:
            reason = "above threshold"
        print(tid, amount, reason)
if not found:
    print("No suspicious transactions")
```

Case Study 3: Student Marks Matrix Analysis (#42)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Analyze marks for multiple students and subjects; calculate student totals and averages, subject averages, and top performer.

Objectives

Represent marks as a nested list and process its rows and columns using nested loops.

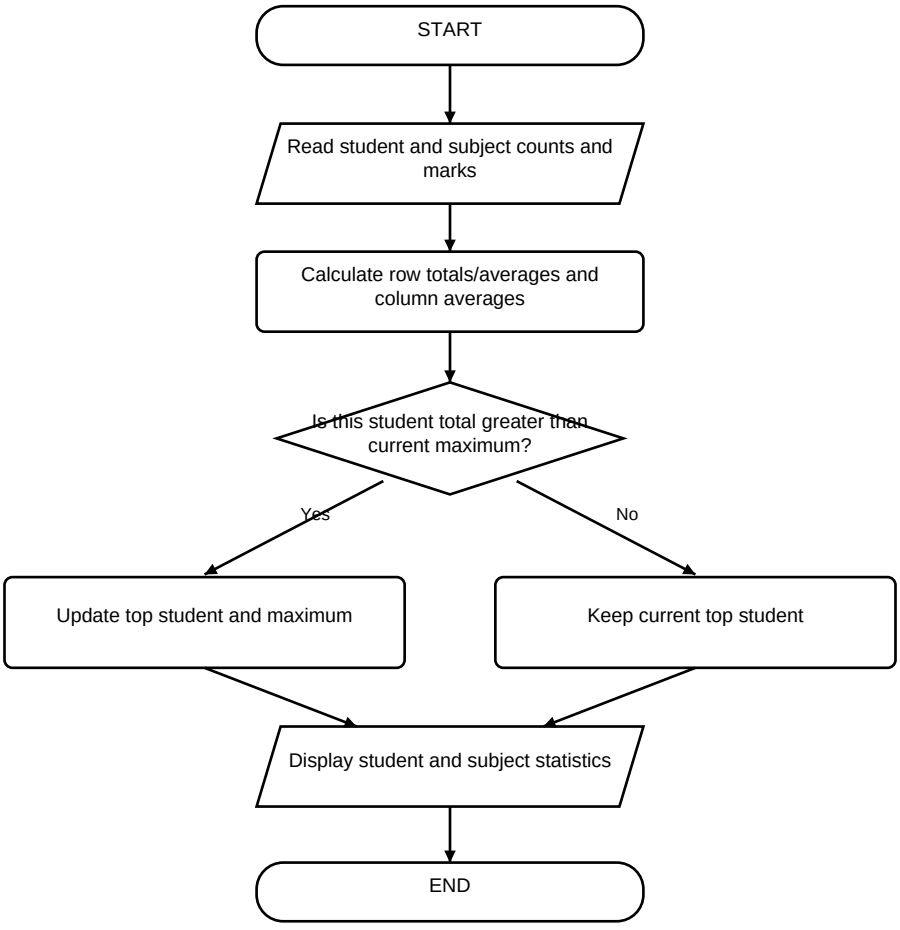
Assumptions

At least one student and subject; marks are integers from 0 to 100; ties go to the first student entered.

Algorithm

Read dimensions and matrix; calculate each row total and average; calculate each column average; identify greatest row total.

Flowchart



Case Study 3: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
INPUT students, subjects; build marks matrix
FOR each row: calculate total and average
FOR each column: calculate subject average
SET maximum = -1
FOR each row: if total > maximum, update maximum and top student
DISPLAY results
```

Test Cases

[[80,70],[90,90]] -> totals 150, 180; subject averages 85, 80; student 2 top.

[[50,50],[50,50]] -> tie; student 1 top.

[[0],[100]] -> student 2 top.

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

Nested lists model a marks table; row and column traversals provide student and subject summaries.

Case Study 3: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("Student Marks Matrix Analysis")
students = int(input("Number of students: "))
subjects = int(input("Number of subjects: "))
marks = []
for i in range(students):
    row = []
    for j in range(subjects):
        row.append(int(input("Mark: ")))
    marks.append(row)
best = -1
top = 0
for i in range(students):
    total = 0
    for j in range(subjects):
        total += marks[i][j]
    print("Student", i + 1, "total", total,
          "average", round(total / subjects, 2))
    if total > best:
        best = total
        top = i + 1
for j in range(subjects):
    total = 0
    for i in range(students):
        total += marks[i][j]
    print("Subject", j + 1, "average",
          round(total / students, 2))
print("Top student:", top, "total:", best)
```

Case Study 4: Tuple-Based Employee Shift Conflict Detection (#48)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Detect overlapping shifts assigned to the same employee.

Objectives

Store shifts as tuples, unpack values, and compare intervals.

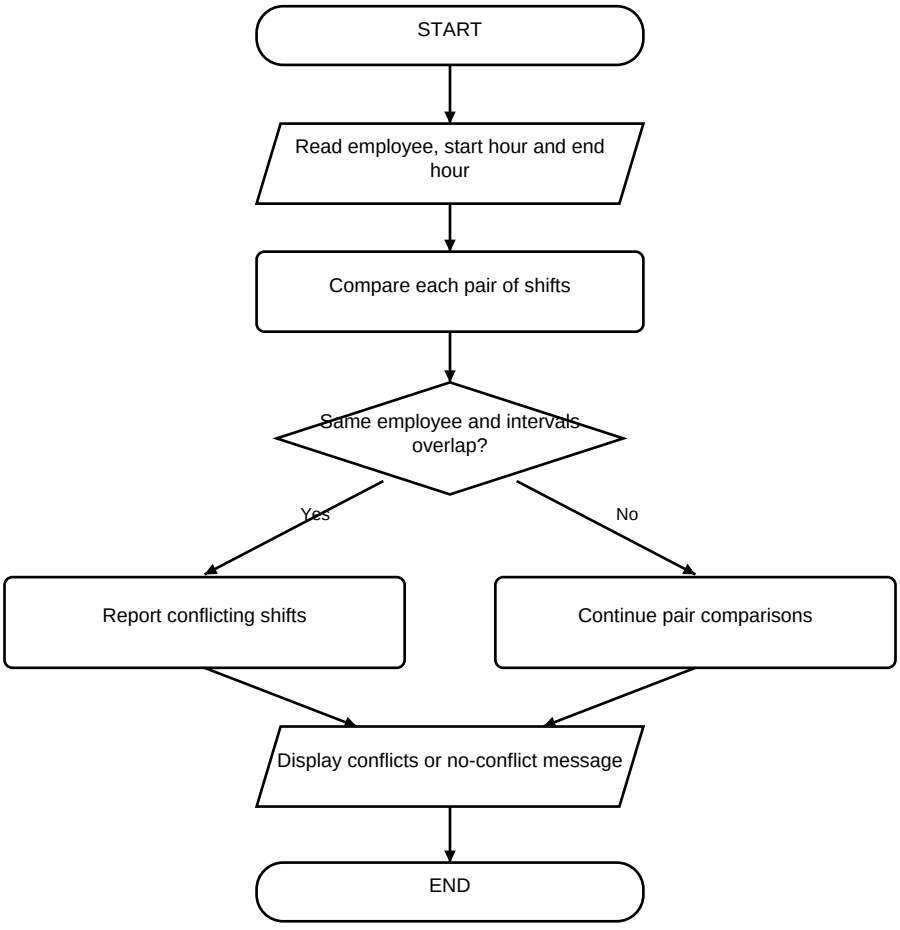
Assumptions

Shifts occur on the same day with start < end; endpoint-touching shifts do not overlap.

Algorithm

Read shift tuples; compare each unique pair; report same-employee overlaps; report if none.

Flowchart



Case Study 4: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
INPUT n; repeat n times: read employee,start,end; store tuple
conflict = false
FOR i from 0 to n-1
FOR j from i+1 to n-1
unpack both tuples
IF same employee AND start1 < end2 AND start2 < end1: display; conflict=true
IF not conflict: display no conflicts
```

Test Cases

Asha 9-13 and Asha 12-15 -> conflict.

Dev 9-12 and Dev 12-15 -> no conflict.

Mia 9-12 and Noah 10-11 -> no conflict.

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

Tuple unpacking and pairwise interval comparisons provide a simple shift-conflict check.

Case Study 4: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("Employee Shift Conflict Detection")
n = int(input("Number of shifts: "))
shifts = []
for i in range(n):
    name = input("Employee: ")
    start = int(input("Start hour: "))
    end = int(input("End hour: "))
    shifts.append((name, start, end))
conflict = False
for i in range(len(shifts)):
    name1, start1, end1 = shifts[i]
    for j in range(i + 1, len(shifts)):
        name2, start2, end2 = shifts[j]
        if name1 == name2 and start1 < end2 and start2 < end1:
            print(name1, "conflict:", start1, end1, start2, end2)
            conflict = True
if not conflict:
    print("No conflicts found")
```

Case Study 5: Food Waste Monitoring System (#53)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Calculate leftovers and waste percentages over several days and identify days exceeding an acceptable limit.

Objectives

Validate meal counts; calculate leftovers and percentages; flag excessive waste.

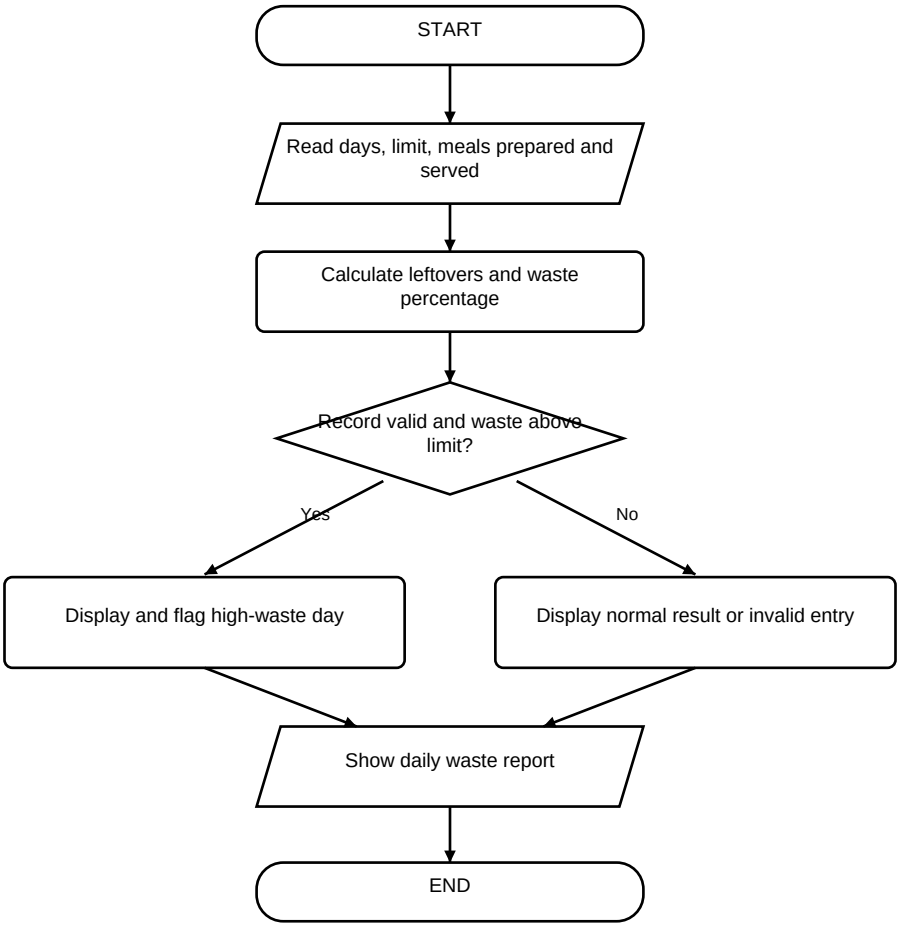
Assumptions

Prepared > 0 and 0 <= served <= prepared; invalid records are rejected; threshold comparison is strictly greater than.

Algorithm

Read day count and limit; for each day validate prepared/served; calculate leftover and waste percent; flag when over limit.

Flowchart



Case Study 5: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
INPUT days, limit
REPEAT days times: INPUT prepared, served
IF prepared <= 0 OR served < 0 OR served > prepared: report invalid
ELSE leftover=prepared-served; percent=leftover*100/prepared; display
IF percent > limit: flag
```

Test Cases

100 prepared, 80 served, limit 15 -> 20 leftovers; 20% flagged.

200, 190, limit 10 -> 10 leftovers; 5%, not flagged.

0, 0 -> invalid.

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

Arithmetic and threshold checks make daily food waste visible and highlight days that need attention.

Case Study 5: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("Food Waste Monitoring")
days = int(input("Number of days: "))
limit = float(input("Waste limit percent: "))
for day in range(1, days + 1):
    prepared = int(input("Meals prepared: "))
    served = int(input("Meals served: "))
    if prepared <= 0 or served < 0 or served > prepared:
        print("Invalid record")
    else:
        left = prepared - served
        percent = left * 100 / prepared
        print("Day", day, "leftover", left,
              "waste percent", round(percent, 2))
        if percent > limit:
            print("Above acceptable limit")
```

Case Study 6: Smart Canteen Order and Stock Management (#57)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Process orders, calculate bills, update stock, reject invalid/unavailable quantities, and summarize sales.

Objectives

Use parallel lists for menu, prices and stock; validate orders; update inventory.

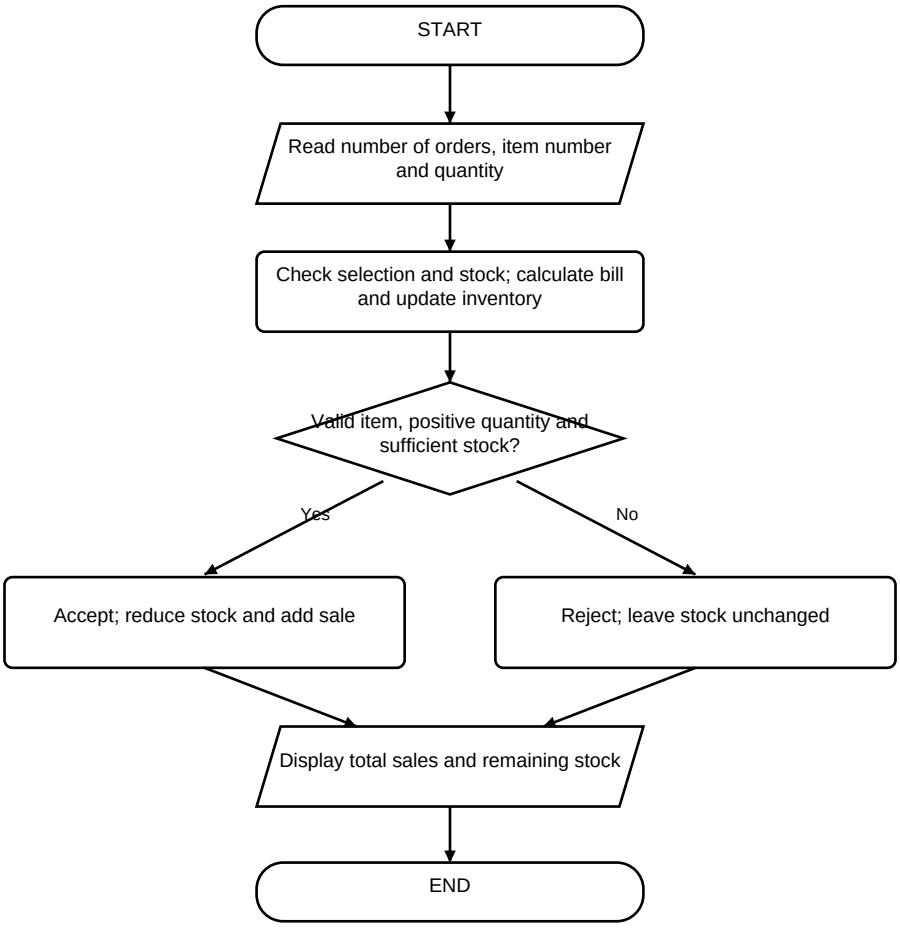
Assumptions

Three fixed items; one item per order; no taxes or discounts; stock is non-negative.

Algorithm

Initialize menu/price/stock; read order count; display menu per order; validate selection and quantity; accept and update or reject; display sales and stock.

Flowchart



Case Study 6: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
SET item,price,stock lists; sales=0
REPEAT order count
DISPLAY menu; INPUT choice, quantity
IF invalid choice OR quantity<=0: reject
ELSE IF quantity>stock[index]: reject
ELSE bill=price[index]*quantity; reduce stock; add sales
DISPLAY sales and remaining stock
```

Test Cases

Sandwich quantity 2 -> accept, Rs.100, stock 8.

Juice quantity 20 -> reject; stock unchanged.

Item 4 or quantity 0 -> reject.

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

Parallel lists keep product data aligned, and validation protects stock from invalid orders.

Case Study 6: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("Smart Canteen")
items = ["sandwich", "juice", "fruit"]
prices = [50, 30, 20]
stock = [10, 12, 15]
sales = 0
orders = int(input("Number of orders: "))
for order in range(orders):
    for i in range(len(items)):
        print(i + 1, items[i], prices[i], stock[i])
    choice = int(input("Item number: "))
    quantity = int(input("Quantity: "))
    if choice < 1 or choice > len(items) or quantity <= 0:
        print("Rejected: invalid choice or quantity")
    else:
        index = choice - 1
        if quantity > stock[index]:
            print("Rejected: insufficient stock")
        else:
            bill = prices[index] * quantity
            stock[index] -= quantity
            sales += bill
            print("Accepted; bill:", bill)
print("Sales:", sales)
for i in range(len(items)):
    print(items[i], "stock:", stock[i])
```

Case Study 7: Multi-Product Price Fluctuation Analysis (#63)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Calculate percentage price changes, flag changes beyond a threshold, and identify the largest absolute fluctuation.

Objectives

Store old and new prices in lists; calculate signed percentage change; compare magnitudes.

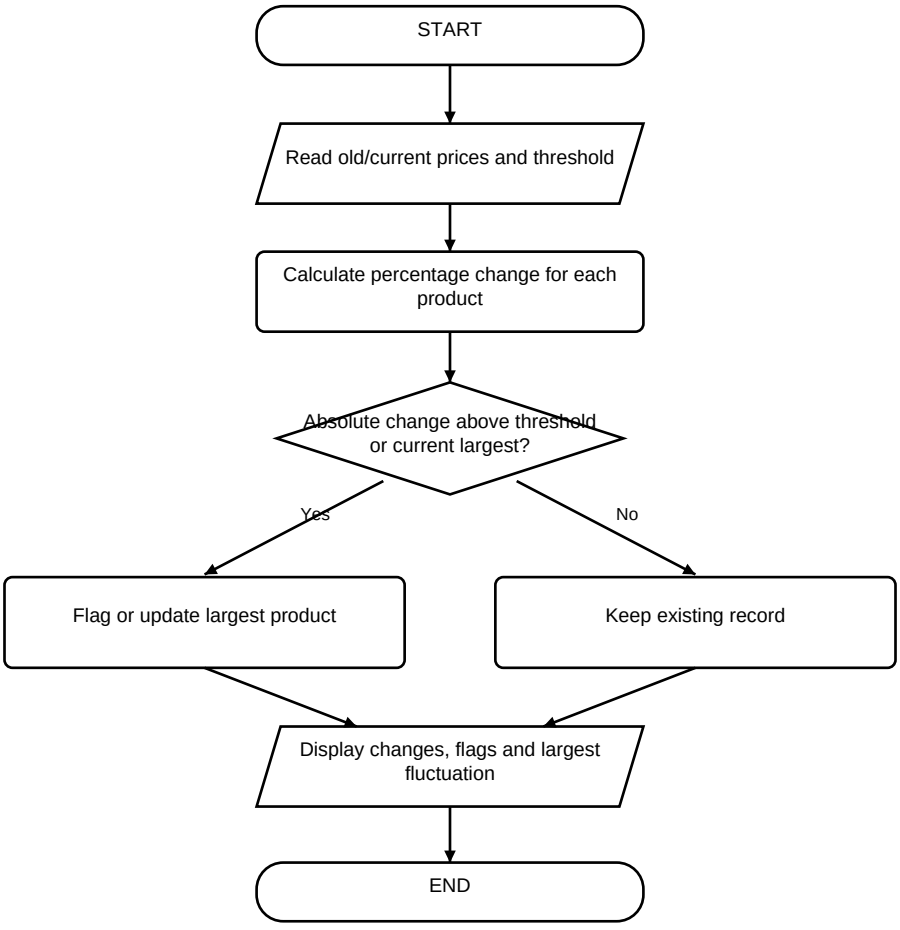
Assumptions

Product list fixed; old price > 0; new price >= 0; increases and decreases count by absolute percent change.

Algorithm

Read prices for products and threshold; validate; calculate change; flag large magnitude; track largest magnitude.

Flowchart



Case Study 7: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
INPUT old and new prices for products; INPUT threshold
largest = -1
FOR each product
IF old<=0 OR new<0: report invalid
ELSE change=(new-old)*100/old
display change; if abs(change)>threshold flag
if abs(change)>largest: update largest/product
DISPLAY largest
```

Test Cases

Rice 50->60, oil 100->90, sugar 40->40; limit 15 -> rice +20% flagged/largest.

Rice 50->40, oil 100->120; limit 15 -> oil +20% largest.

Old price 0 -> invalid.

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

List processing compares multiple price histories and detects unusually large changes in either direction.

Case Study 7: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("Product Price Fluctuation")
products = ["rice", "oil", "sugar"]
old = []
new = []
for product in products:
    old.append(float(input(product + " previous price: ")))
    new.append(float(input(product + " current price: ")))
limit = float(input("Change threshold percent: "))
largest = -1
largest_name = ""
for i in range(len(products)):
    if old[i] <= 0 or new[i] < 0:
        print(products[i], "invalid prices")
    else:
        change = (new[i] - old[i]) * 100 / old[i]
        print(products[i], round(change, 2), "%")
        if abs(change) > limit:
            print("Above threshold")
        if abs(change) > largest:
            largest = abs(change)
            largest_name = products[i]
print("Largest fluctuation:", largest_name, largest)
```

Case Study 8: Student Academic Risk Classification (#66)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Classify students based on marks, attendance and laboratory scores; report the number of risk factors.

Objectives

Use parallel lists, conditions and a counter to combine academic indicators.

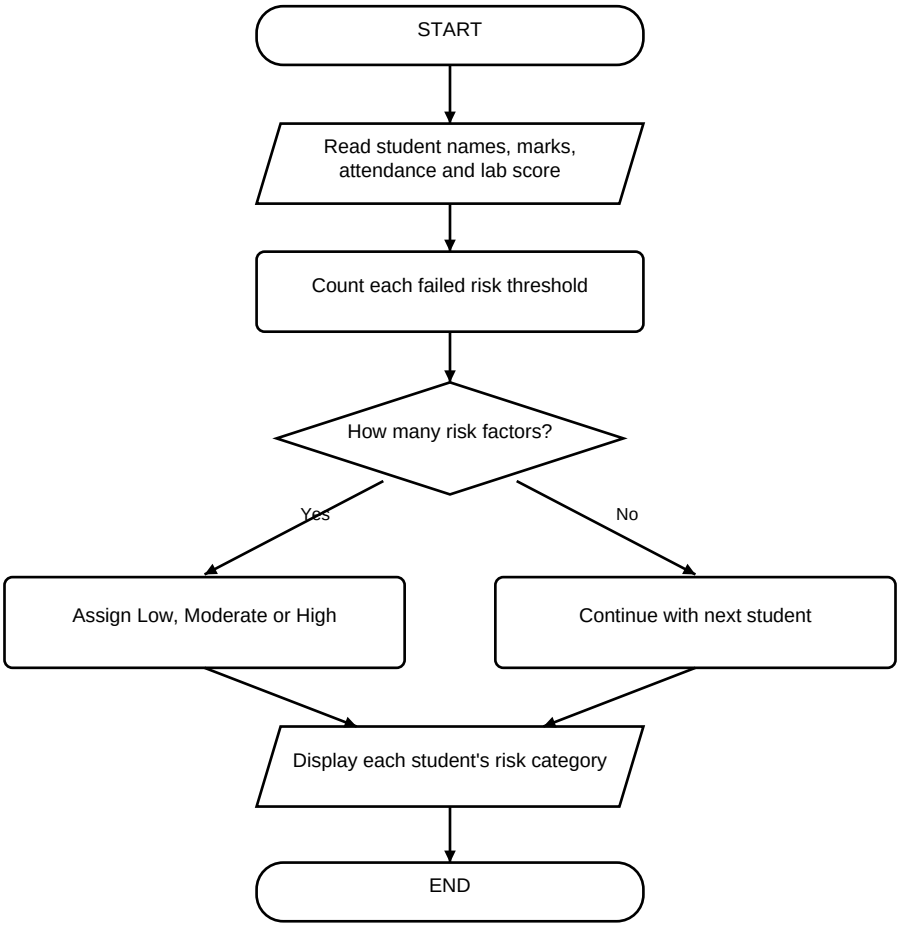
Assumptions

Values 0-100; risk rules: marks<50, attendance<75, lab<50; 0 factors Low, 1 Moderate, 2-3 High.

Algorithm

Read each student's details; count failed thresholds; assign category; display name, category and count.

Flowchart



Case Study 8: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
INPUT n; for each student read name,marks,attendance,lab
FOR each student: risks=0
IF marks<50 add 1; IF attendance<75 add 1; IF lab<50 add 1
IF risks=0 Low; ELSE IF risks=1 Moderate; ELSE High
DISPLAY name, category, risks
```

Test Cases

80,90,70 -> Low (0).

40,90,70 -> Moderate (1).

40,60,30 -> High (3).

50,75,50 -> Low (0; strict thresholds).

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

A risk counter combines several conditions into a clear student classification.

Case Study 8: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("Academic Risk Classification")
n = int(input("Number of students: "))
names = []
marks = []
attendance = []
labs = []
for i in range(n):
    names.append(input("Name: "))
    marks.append(float(input("Average marks: ")))
    attendance.append(float(input("Attendance percent: ")))
    labs.append(float(input("Lab score: ")))
for i in range(n):
    risk = 0
    if marks[i] < 50:
        risk += 1
    if attendance[i] < 75:
        risk += 1
    if labs[i] < 50:
        risk += 1
    if risk == 0:
        category = "Low"
    elif risk == 1:
        category = "Moderate"
    else:
        category = "High"
    print(names[i], category, "risk factors:", risk)
```

Case Study 9: Campus Festival Registration and Financial Audit (#74)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Reconcile registrations, fees, sponsorship and expenses; detect repeated IDs and calculate final balance.

Objectives

Store tuple records; identify duplicate IDs; calculate collected/expected fees and balance.

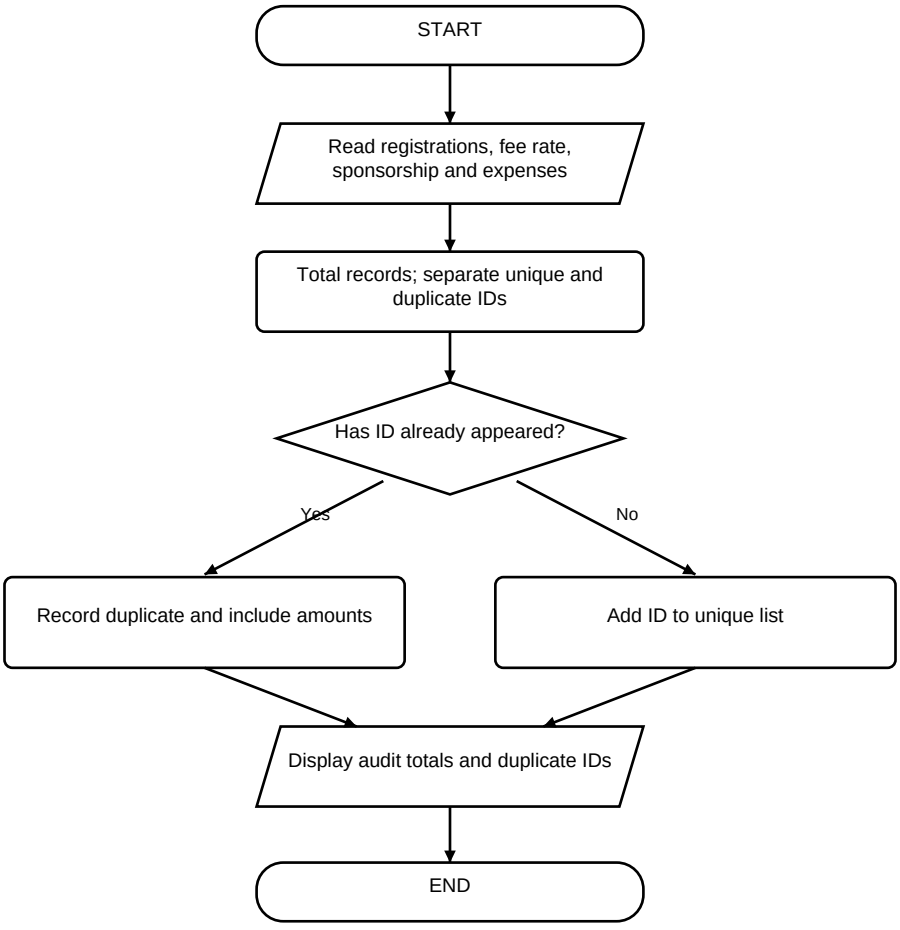
Assumptions

Expected fee is registration fee times unique IDs; all recorded payments/sponsorship are included in actual totals; balance=fees+sponsorship-expenses.

Algorithm

Read registration tuples and expenses; total amounts; collect unique and duplicate IDs; calculate expected fees and balance; display report.

Flowchart



Case Study 9: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
INPUT records (ID, paid, sponsor); INPUT expenses
FOR each record: add amounts to totals
IF ID seen: record duplicate ELSE append unique ID
expected=unique_count*fee_rate
balance=fees+sponsorship-expenses
DISPLAY totals, duplicates, balance
```

Test Cases

A and B pay 100 each; fee 100, sponsor 50, expenses 150 -> 2 unique, balance 100.

A pays 100 twice; fee 100, expense 50 -> duplicate A; actual fees 200; balance 150.

No registrations; expense 40 -> balance -40.

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

Tuple records and ID lists support a simple audit of participation and event finances.

Case Study 9: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("Campus Festival Financial Audit")
n = int(input("Number of registrations: "))
rate = float(input("Fee per participant: "))
records = []
for i in range(n):
    sid = input("Participant ID: ")
    paid = float(input("Fee paid: "))
    sponsor = float(input("Sponsorship: "))
    records.append((sid, paid, sponsor))
expenses = 0
for i in range(int(input("Number of expenses: "))):
    expenses += float(input("Expense: "))
unique = []
duplicates = []
fees = 0
sponsors = 0
for sid, paid, sponsor in records:
    fees += paid
    sponsors += sponsor
    if sid in unique:
        if sid not in duplicates:
            duplicates.append(sid)
    else:
        unique.append(sid)
expected = len(unique) * rate
balance = fees + sponsors - expenses
print("Unique participants:", len(unique))
print("Fees collected:", fees, "Expected:", expected)
print("Sponsorship:", sponsors, "Expenses:", expenses)
print("Balance:", balance, "Duplicate IDs:", duplicates)
```

Case Study 10: Environmental Monitoring Data Analyzer (#84)

Problem, objectives, assumptions, algorithm and flowchart

Problem Statement

Analyze readings at locations over time; compute average, minimum and maximum; flag observations outside limits.

Objectives

Use nested lists and loops for statistics and threshold detection.

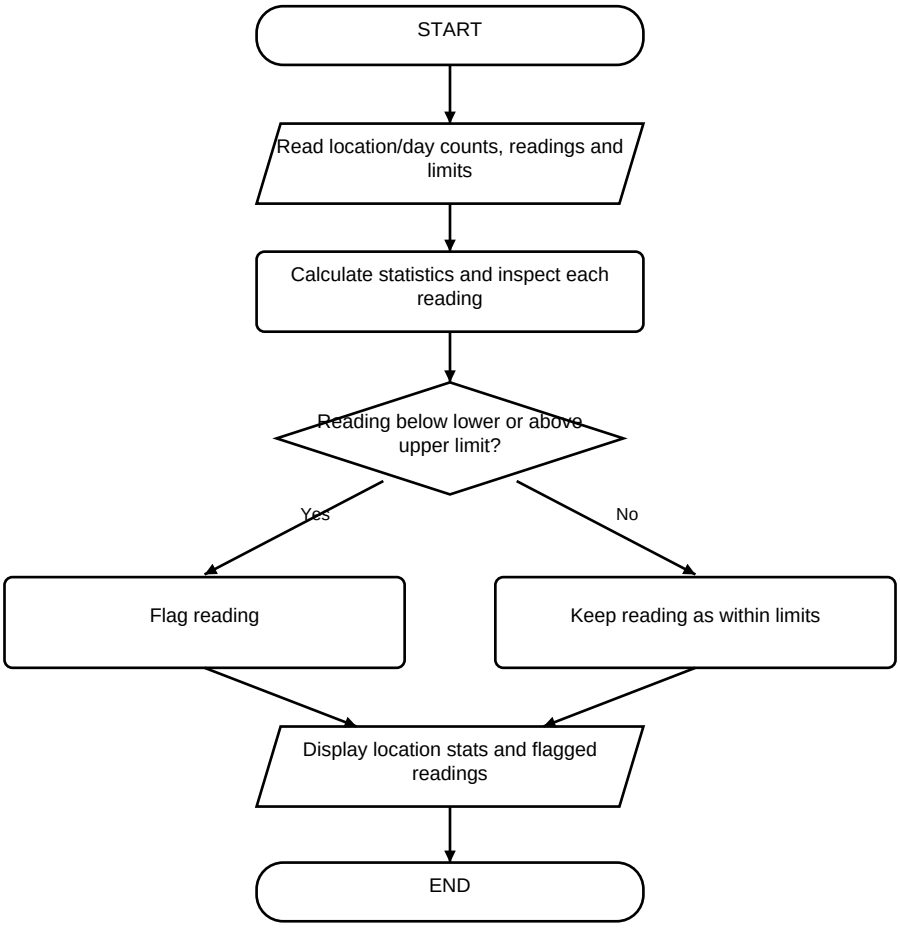
Assumptions

At least one location and reading; lower and upper limits inclusive; numeric readings.

Algorithm

Read dimensions and matrix; read limits; for each location calculate stats; check and flag each value outside range.

Flowchart



Case Study 10: Pseudocode and Test Plan

Pseudocode, test cases, screenshot instruction and conclusion

Pseudocode

```
INPUT locations, days; build nested reading list
INPUT lower, upper
FOR each location: total/min/max; average=total/days; display
FOR each reading: if value<lower OR value>upper flag
```

Test Cases

10,12,14; limits 8..15 -> average 12, min 10, max 14; none flagged.

5,10,18; limits 8..15 -> readings 5 and 18 flagged.

8,15; limits 8..15 -> none flagged.

Output Screenshot

Run the program using a normal and an edge-case test. Capture the console showing inputs and complete output, then place the genuine screenshot in this case folder's output/ directory.

Conclusion

Nested lists organize measurements by location and repeated calculations reveal out-of-range readings.

Case Study 10: Python Source

Save this listing as main.py. Student should run and understand the code.

```
print("Environmental Monitoring Analyzer")
locations = int(input("Number of locations: "))
days = int(input("Readings per location: "))
data = []
for i in range(locations):
    row = []
    for j in range(days):
        row.append(float(input("Reading: ")))
    data.append(row)
lower = float(input("Lower limit: "))
upper = float(input("Upper limit: "))
for i in range(locations):
    total = 0
    low = data[i][0]
    high = data[i][0]
    for j in range(days):
        value = data[i][j]
        total += value
        if value < low:
            low = value
        if value > high:
            high = value
    print("Location", i + 1, "average", round(total / days, 2),
          "min", low, "max", high)
    for j in range(days):
        if data[i][j] < lower or data[i][j] > upper:
            print("Out of range reading", j + 1, data[i][j])
```